

*The ordered work queue lives in **PRIORITIES.md** (opened 2026-08-04). This file is domain analysis — what a set is, which slots are genuinely open, and how to size an experiment over them. Read this to design a test; read PRIORITIES.md to find out what to test next.*

Set — everything about one Pokémon other than which Pokémon it is: its four **moves**, its **item**, its **ability**, its **nature**, and its **EV spread**. Standard VGC usage, and the reason this document says "set" rather than listing those five things every time. "Incineroar's set" is a complete answer to *what is that Incineroar running*.

Locked vs flex slots. Measured from Smogon, most Pokémon have very little genuinely open. Only the flex slots are worth varying in an experiment:

species	locked (>=85% of sets)	genuinely open
Incineroar	Fake Out 100%, Parting Shot 95%, Flare Blitz 91%	1 slot — Throat Chop 51% / Darkest Lariat 31% / Protect 12% / Will-O-Wisp 5%
Whimsicott	Tailwind 99%, Moonblast 92%	2 slots
Garchomp	Dragon Claw 92%	3 slots

So "test every set" is smaller than it sounds. For Incineroar it is four candidates for one slot, not a free choice of four moves from a learnset.

Gauntlet — the fixed list of opponent teams every build is tested against. Same teams, same order, same seeds for every build, so the comparison is paired and the matchup difficulty cancels.

Points — percentage points of win rate. A build at 53% against one at 50% differs by 3 points. Halving the difference you want to detect roughly quadruples the games required.

What is known-available and not yet used, and what the project is ultimately for. Kept short and concrete: anything here should be actionable without re-deriving why.

The scoring bot exists (v3.7.0). It reads the board, and both gaps it was built for roughly halved. It is one ply and it does not decide switches, so it is a first version, not a finished one — see item 3 for exactly what it does and does not do.

out of sample, 600 seed-matched battles	prior (was)	score (now)	real
moves that were super effective	9.71%	14.91%	21.37%
moves that outright failed	9.68%	6.34%	2.47%
moves that hit an immune target	4.30%	2.92%	1.91%
moves that were Protect-type	21.62%	16.71%	13.87%
switches per game	8.31	8.38	10.67

What is next, in order. (1) **Switching** — the one decision still inherited untouched, and the last big behavioural gap at 8.38 against 10.67. It is also what the residual mega gap was traced to. (2) The **head-to-head gate**, item 4, which is now meaningful because there are two policies to compare. (3) `build_lab` re-run: every win rate in it was measured against a pilot that could not read the board, so they are all provisional and none has been re-measured yet.

The two findings from the fit that generalise past it. The behaviour clone alone predicts human clicks *worse than choosing uniformly at random*, and the fit weights it at only +0.25 — it is far too confident about the popular move. And the biggest learned effects are not about damage: they are the "this move is already dead" terms. Reading the board is mostly about **not clicking moves that cannot work**.

Settled tonight, do not redo:

- Set diversity is CLOSED — measured at matched n we are MORE varied than the ladder (12.75 v 12.17).
- `move-priors.json` was learned from bot games (7:1) and is now clean-filtered. Distributions moved up to 8 points. This was the largest real defect in the project.
- All five live models now either filter or carry a `RAW-STORE-OK` declaration saying why they must not. `engine/selftest.js` enforces it and is RED with 18 remaining — all non-live.
- `build_lab` had three statistical defects (self-comparison, nested-prefix factorial, pilot airtime bias). All fixed. See `docs/DEFENSE.md`.
- Megas 74% -> 80%; the residual is a lead/switch issue downstream of the scoring bot.

Runs itself now — do not hand-maintain any of it: `move-priors`, `bring-priors`, `data/live.js`, ORIENTATION's figures, the CHOMP re-run trigger, regulation-rotation detection, and the archive. All hourly, all derived.

~**2026-08-15**: the clean corpus passes 10,609 games and `build/triggers.js --fire` re-runs the CHOMP proof automatically. Reg M-B is scheduled to end 2026-09-02, so it resolves with ~18 days to spare at 1.9x the required accrual rate.

Known and not fixed:

1. `pory.py` reports "beats coin AND material heuristic" while its own CI [0.6264, 0.7321] spans the coin's 0.6931. Same overclaim pattern `build_lab` had. Check before quoting it.
2. Generated sets under-produce Smogon's locked moves by 2.6 points. The fix is per-move Bernoulli sampling at the Smogon set-rate with repair to four — NOT force-inclusion, which was tried, collapsed Kingambit's variety to zero, and was reverted (`docs/DEFENSE.md` §2).
3. `build_lab` tests ONE host team against 40 opponents and does not say so in its output.
4. 18 non-live files still read the store undeclared.
5. Uniform piloting is unbiased across moves only if they are exchangeable in time. Setup moves are not, so setup builds are currently understated. **This was previously recorded as "dissolved by the scoring bot" — it is not, and that entry was optimistic.** The scoring bot keeps `uniformFor` for the species under test, deliberately, because `build_lab` needs equal airtime per arm. So the *opponents* now read the board and the *tested species* still does not, and setup builds are still understated. Dissolving it properly means giving the tested species a board-aware pilot and accepting unequal airtime, or scoring the arms some other way — an open design question, not a patch.
6. Every `build_lab` win rate on record was measured against the old board-blind pilot. None has been re-run against the scoring bot, so all of them are provisional.

ALAKAZAM climbs the real ladder, high. That is the proof.

A person should also be able to sit down and play it — but the ladder is the *test*, and it is the right one for a reason worth stating: **it is the only measure in this project that self-play cannot fake.** Every internal number can be gamed by a bot that gets good at beating itself. Ladder rating is external, adversarial, and measured by people trying to win.

It is also, precisely, the transfer test at full scale. A model that has overfit to its own play will climb to a plateau and stop; one that has learned Pokemon will keep going.

Honest about the bar: VGC-Bench beat a professional **on a single fixed team** and degraded as team variety rose. The PokeAgent Challenge (NeurIPS 2025, 100+ teams) concludes the general problem is **unsolved**, with large gaps between LLM agents, RL agents, and strong humans. So "stupidly high" is a research-grade target, not an engineering one — and the fixed-team scoping in `docs/THEORY.md` §5 is the version that is actually reachable first.

What it requires that we do not have:

- a policy that decides in **real time**, at human pace
- a Showdown connection — a bot account or custom client. `web/replay.html` already proves we speak the protocol in one direction
- **ladder discipline**: a rating is only meaningful over hundreds of games, and every one is public and permanent

Everything below is scaffolding for that. When a decision is ambiguous, the question is which option moves a real rating.

All of this is already downloaded monthly by `.github/workflows/smogon-stats.yml` and parsed by `engine/smogon_priors.js`. Nothing here needs new collection; it needs wiring.

data	what it gives	what it would replace
Checks and Counters	per species, what beats it, with a win rate, a 95% interval , and how the matchup ends (KOed% / switched%)	GURU's own archetype matrix, which finds zero decisive matchups on ~1,700 clean games. Smogon computes the same thing over 1.16M battles and ships the interval with it.
Teammates (full list)	partner frequency as points above/below chance — e.g. Garchomp + Whimsicott +42.0	the k-means archetype clustering, which is capped at K≈6 by sample size and scores silhouette 0.114 (essentially no structure)
Viability Ceiling	highest GXE any team using that species reached	nothing — this is new information, a skill-ceiling signal per species
Raw vs Real counts	Raw = teams containing it, Real = battles it actually appeared in. Ratio ≈ how often it is BROUGHT	our own bring rates, which are conditioned on what a replay revealed
Spreads	nature + EV distribution per species	already used for spreads; not yet used to model what a spread implies about role
Mega formes as separate species	the stone rate, and the X/Y split	now wired (<code>megaInfo</code>), listed here as the precedent for the rest

Why this matters more than it looks. Their sample is 1,163,315 battles, computed server-side over every team played. Ours is ~1,700 clean replays, uploaded voluntarily, and only shows what a battle revealed. For anything about *what teams look like*, Smogon is strictly better and we should stop deriving it ourselves. For *per-turn behaviour*, they publish nothing and our store is the only source. The two answer different questions; neither replaces the other.

Caveat to carry forward: the cutoff files are weightings, not filters. All four report the same 1,163,315 battles; the average weight falls from 1.000 (cutoff 0) to 0.078 (1630) to 0.004 (1760). So the *effective* sample at 1760 is ~4,650 battles — barely larger than our own store, and noisy. Default to 1630 (~91,000 effective) and never quote 1760 as authoritative.

Ordered by what blocks what.

1. **Set diversity — CLOSED. The gap does not exist.** (*kept, because the way it was wrong is the useful part*)

Final measurement, at matched sample size across 76 shared species: **we produce 13.2 distinct sets per species against a real 11.3**. We are slightly MORE varied than the ladder. There is nothing to fix.

It was reported as a defect three times, at 23-against-51, then 9.8-against-13.0, then as a thin-candidate-pool problem, and every version was a measurement artifact:

- `realism_report` capped the generated corpus with `--limit` but read the real one in full, and distinct-counts grow with n mechanically. On 292 games against 13,249 it printed 5.7 against 52.0 and flagged itself "large" every run.
- Real games reveal fewer moves per set (1.40 of 4 against our 1.83), so more partial views of the SAME set count as distinct on the real side.
- The thin-pool explanation was tested directly: correlation between our candidate-pool size and the gap is **0.04** across 28 species. None.

The lesson, which generalises past this item: every one of those was a denominator or sample-size mismatch between the two corpora, and the same class of error previously invented a switching defect (23.3 against 46.7 per 100 moves, which is 4.3 against 5.7 per game). Before believing any generated-vs-real gap, check that both sides had the same opportunity to produce it.

What remains true and is now item 1a: we produce the single most common set 48% of the time against a real 44%. Small, real, and not what any of the above was about.

1. **~~Set diversity, RE-SCOPED~~** (*superseded by the above; text kept for the record*)

The realism report showed 23 distinct sets per species against 51 real, and that was called the largest remaining gap. It was mostly a measurement artifact. Two confounds, both inflating the real side: real games reveal FEWER moves per set (1.40 of 4 against our 1.83), so more partial views of the SAME set count as distinct; and real supplies 4.6x more sets (114,670 against 24,822), and distinct-counts grow with sample size mechanically.

Compared fairly — fully-revealed sets only, equal sample size per species — it is **9.8 against 13.0**, a 25% gap rather than 55%.

And it is concentrated exactly where the locked/flex structure predicts. Pokemon whose sets are locked already match: Rotom-Wash 12 v 12, Corviknight 14 v 15, Politoed 10 v 8 (ours is MORE varied). The gap lives in species with several genuinely open slots: Garchomp 14 v 35 (three open slots), Toxapex 4 v 11.

So the fix is not "make everything more varied" — most Pokemon do not run varied movesets and ours are right.

The stated cause was then tested and is WRONG. The claim was that our candidate pool is too thin. Measured: the correlation between our pool size and the diversity gap is **0.04** over 28 species — none. Every species has ~7.8 candidates, Rotom-Wash and Garchomp alike, and Rotom-Wash has no gap. Pool size is not the mechanism.

What IS measurable, on the same 28 species: we produce the single most common set **48%** of the time against a real **44%**, and the excess is concentrated exactly where the gap is — Toxapex +18 points, Whimsicott +15, Garchomp +11, Kingambit -2 (we are slightly more varied than reality). It is mode collapse in the sampler, not a missing pool.

There is also a **floor** on how well this can ever be fixed, now quantified. Smogon lists moves individually to about 1% and buckets the rest as "Other", which runs 15-20% for nearly every species. Since a set is four moves, $1 - (1 - \text{other}/400)^4$ says **~17% of real sets contain a move no prior of ours can propose**. Kingambit is the exception at 3%. Reported by `engine/set_space.js`; do not chase the last few points of diversity without accounting for it.

1. **Megas — was 74% against a real 93%, now 80%. Partly fixed; the stated cause was wrong.**

The claim here was "real players bring their mega". **They do not.** Smogon's raw counts (teams containing it) against real counts (battles it played) say a mega forme is brought **0.90x** as often as a non-mega. Mega-capable species were already being brought at 88.6% against a real 92.6%, so the bring was close to right all

along.

Three other candidates were measured and ruled out: team generation puts **1.54** stones on a team against the **1.58** Smogon implies, and stones match their holder 99.6% of the time; raising the player's form-change probability from 0.85 to 1.0 moved the rate **0.7 points**; and Champions has no Terastallization, so the tera-before-mega priority inside `RandomPlayerAI` never fires (measured 0.00 teras per game).

The cause was the **lead**, not the bring — a stone holder in the back of a game that ended before it came out. `bring_priors.js` now measures 77.5% of real sides mega and 57.7% of megas are a lead, and `chooseTeamPreview` aims at both. **74% -> 80.1%** on 300 seed-matched games.

Still open: 80% against 93%, and 1.07 against 1.58 megas per game. Form-change 1.0 on top of the fix gives 80.5%, so that knob is saturated. Most likely the back-slot holder still reaches the field less than a human's, consistent with our 4.3 switches per game against 5.7 — which makes this downstream of item 3, the scoring bot.

1. The scoring bot — BUILT in v3.7.0, and it is one ply.

`engine/board.js` reconstructs the board a decision was made against; `engine/fit_policy.js` fits (move, target) features to 48,538 real human clicks from 2,240 clean open-sheet games by conditional logit; `engine/magnemite.js` plays it. `mew.js --policy score`.

Open sheets are used because they are the only corpus where the **choice set** is known rather than guessed — a normal replay reveals only the moves that were *used*, so alternatives reconstructed from revelation are biased by revelation itself.

Held out by game: logL/decision **-1.6006** and top-1 **33.6%**, against the behaviour clone's -1.9302 and 27.1%. The realism-report numbers are in the STATE block above and in CHANGELOG 3.7.0.

Most of the win was aiming, not move choice. `RandomPlayerAI` picks which foe to hit with `prng.random(2)` *before* `chooseMove` is called, so the target was a coin flip however good the move was — and in doubles aiming is most of what "super effective" means.

What it still does not do, stated so nobody assumes otherwise:

- it does **not** decide switches (inherited unchanged — item 3a below),
- it runs **no damage calculation**; base power and type effectiveness stand in for one,
- it has **no model of the opponent's move**, so it cannot read a Protect or bait a switch,
- it is **one ply**. No search.

Two honest caveats. The weights are fitted on open-sheet games, where both players see everything and hedge less than on the closed ladder. And some clicks could not be matched to a candidate and were dropped — **2.94% as of 2026-08-02**, down from roughly four times that.

This entry used to say the largest cause was redirection (Follow Me, Rage Powder). **It is 1.60%** (`engine/redirect_audit.js`). Nobody had measured it; three documents cited each other until it read as settled. The real causes were a foe **switching in on the same turn** (44.4% — a human aims at a slot, the store records a species, and switches resolve first), an **in-battle forme change** with no sheet entry (19.7%), and a **mirror collapsing the two team sheets** (16.4%). All are fixed in `engine/click_match.js`. Redirection's real cost is a MISLABELLED target on 1.55% of clicks, which the store cannot fix because the chosen target was never recorded.

3a. **Switching — now the largest behavioural gap.** 8.38 switches per game against a real 10.67, and the decision is still `RandomPlayerAI` 's. It is also where the residual mega gap was traced to (item 2), and the natural next application of exactly the machinery item 3 just built: the board is already tracked, and "switch" is one more candidate in the same choice set.

1. **Head-to-head gate.** New bot versus old, a few thousand games. If an iteration does not beat the previous one above chance, stop and find out why rather than looping on a broken crank. **Now meaningful**, because as of 3.7.0 there are two policies to compare and `selftest.js` already refuses to bless a refit that scores worse than the clone it replaces.
2. **The transfer test** — train on self-play, evaluate on real games. Never completed. The only check in the project that is not self-referential.

-
- **July's Smogon data flowing in end to end.** The month auto-detects (`latestMonth()` scans the directory) and no month is hardcoded, but this has not been run against a genuinely new month.
 - **The forme audit is done for the families we have seen** (mega naming, Sinistcha, Maushold, Vivillon, Floette-Eternal). It should be re-run whenever a new regulation adds formes, because the failure is silent: a name that matches nothing produces an empty moveset, not an error.
-

Full write-up with measurements: `docs/FINDINGS-2026-08-01-live-play.md` .

One evening of Will playing MAG in the real Showdown client surfaced nine defects. Seven are fixed. These four are not, and three of them need a refit.

- **A REFIT IS OWED, AND IT BLOCKS THE OTHER THREE.** `board.js:2786` changed feature *semantics* without changing the feature *list*: `allyHit` could not see type immunity, so a Flying partner beside Earthquake read as hit and `spreadFreeBesideAlly` could never fire. Every shipped weight was fitted against the old meaning. `board.js` is required by 40 files; the vector is read by 24.
- **Nothing detects a semantics change under an unchanged feature name.** `magnemite.js:297` compares joined feature names and `:299` compares vector length; both pass when a feature quietly starts meaning something else. Proposed guard: a stored hash of each feature's values over a fixed fixture board, checked at load.
- **MAG cannot see that it is Choice-locked.** No feature among the 56 represents holding an unwanted Choice item. Encore made it switch; a Trick'd Scarf did not. Needs a feature and a refit.
- **No pair feature for "both slots set the same side condition."** Two Light Screens on one turn. `deadSide` (-2.879, the most negative weight in the vector) cannot help, because it asks whether the condition is *already up*. Needs `bothSameSideCondition` and a refit.
- **No support floor on sampled sets** (team generation, not the model). 3,949 of 10,504 distinct sets were seen exactly once — 37.6%. The bot's team builder should sample within `species_sets.cover(sp, 0.80)` . Deliberately NOT global: DITTO's gauntlet must keep the full tail.

Unmeasured change shipped knowingly: the bot now runs with `joint: true` . No H2H has compared joint-on against joint-off for a greedy player. It was enabled because it is the only mechanism that can express slot coordination, not because it is measured better.